注意力就是你所需要的一切

Attention Is All You Need

Ashish Vaswani　Noam Shazeer　Niki Parmar　Jakob Uszkoreit
Llion Jones　Aidan N. Gomez　Łukasz Kaiser　Illia Polosukhin

Google Brain　Google Research　University of Toronto

中文全译版（非官方翻译）　原作发表于 2017 年　arXiv:1706.03762

作者说明： 同等贡献，作者顺序随机。Jakob 提出用自注意力替代 RNN，并启动了对这一想法的评估。Ashish 与 Illia 设计并实现了最早的 Transformer 模型，并深度参与了本工作的各个方面。Noam 提出缩放点积注意力、多头注意力和无参数位置表示，也成为另一位几乎参与所有细节的人。Niki 在原始代码库和 Tensor2Tensor 中设计、实现、调优并评估了无数模型变体。Llion 也试验了新的模型变体，负责最初的代码库以及高效推理和可视化。Lukasz 和 Aidan 用无数个漫长的日夜设计并实现了 Tensor2Tensor 的各个部分，取代了早期代码库，大幅改进了结果并极大加速了研究。Aidan 的工作完成于 Google Brain 任职期间；Illia 的工作完成于 Google Research 任职期间。

复制许可说明： 在适当署名的前提下，Google 允许仅为新闻或学术作品复制本文中的表格和图。

摘要　主流的序列转导模型基于复杂的循环神经网络或卷积神经网络，通常包含编码器和解码器。性能最好的模型还通过注意力机制连接编码器与解码器。我们提出了一种新的简单网络架构——Transformer，它完全基于注意力机制，彻底摒弃循环与卷积。在两个机器翻译任务上的实验表明，这些模型不仅质量更优，而且更易于并行化，训练所需时间也显著减少。我们的模型在 WMT 2014 英译德任务上取得 28.4 BLEU，比包括集成模型在内的既有最佳结果高出 2 个 BLEU 以上。在 WMT 2014 英译法任务上，我们的模型在 8 块 GPU 上训练 3.5 天后，以 41.8 BLEU 创造了新的单模型最佳结果，而训练成本仅为文献中最佳模型的一小部分。我们还将 Transformer 成功应用于英语成分句法分析，无论训练数据充足还是有限均表现良好，说明它能够很好地泛化到其他任务。

1　引言

循环神经网络，尤其是长短期记忆网络（LSTM）[13] 和门控循环神经网络 [7]，已经在语言建模、机器翻译等序列建模与序列转导问题中牢固确立了最先进方法的地位 [35, 2, 5]。此后，大量研究不断推进循环语言模型和编码器—解码器架构的边界 [38, 24, 15]。

循环模型通常沿输入和输出序列的符号位置分解计算。它们将位置与计算时间步对齐，根据前一隐藏状态 h_{t-1} 和位置 t 的输入生成隐藏状态序列 h_t。这种固有的顺序性质使训练样本内部无法并行化；当序列较长时，这一点尤其关键，因为内存限制会约束跨样本的批处理。

近期研究通过分解技巧 [21] 和条件计算 [32] 显著提高了计算效率，后者还改善了模型性能。然而，顺序计算这一根本限制依然存在。

注意力机制已成为各类任务中强大序列建模与转导模型不可或缺的组成部分，它能够在不考虑依赖项在输入或输出序列中距离的情况下建模依赖关系 [2, 19]。不过，除少数情况 [27] 外，这类注意力机制都与循环网络结合使用。

本文提出 Transformer，一种摒弃循环、完全依赖注意力机制来捕获输入与输出之间全局依赖关系的模型架构。Transformer 能够实现显著更高的并行度；在 8 块 P100 GPU 上仅需训练 12 小时，即可达到新的翻译质量最佳水平。

2 背景

减少顺序计算的目标也是 Extended Neural GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础。这些模型均以卷积神经网络为基本构建块，对所有输入和输出位置并行计算隐藏表示。在这些模型中，关联任意两个输入或输出位置的信号所需的操作数会随位置间距增加：ConvS2S 为线性增长，ByteNet 为对数增长。这使得学习远距离位置之间的依赖更加困难 [12]。在 Transformer 中，这一操作数被降为常数，尽管对注意力加权后的位置取平均会降低有效分辨率；我们通过第 3.2 节所述的多头注意力抵消这一影响。

自注意力有时也称内部注意力（intra-attention），是一种关联单个序列中不同位置、从而计算该序列表征的注意力机制。自注意力已成功用于阅读理解、抽象摘要、文本蕴含以及学习与任务无关的句子表示等多种任务 [4, 27, 28, 22]。

端到端记忆网络基于循环注意力机制，而非沿序列对齐的循环结构；研究表明，它在简单语言问答和语言建模任务上表现良好 [34]。

然而，据我们所知，Transformer 是首个完全依靠自注意力来计算输入与输出表示、而不使用沿序列对齐的 RNN 或卷积的转导模型。以下各节将介绍 Transformer，阐述采用自注意力的动机，并讨论它相对于文献 [17, 18] 和 [9] 等模型的优势。

3 模型架构

大多数有竞争力的神经序列转导模型采用编码器—解码器结构 [5, 2, 35]。编码器将符号表示的输入序列 $(x_1, \ldots, x_n)$ 映射为连续表示序列 $\mathbf{z} = (z_1, \ldots, z_n)$。给定 $\mathbf{z}$，解码器随后逐元素生成符号输出序列 $(y_1, \ldots, y_m)$。在每一步中，模型都是自回归的 [10]：生成下一个符号时，会将此前生成的符号作为附加输入。

Transformer 遵循这一总体架构，其编码器和解码器均使用堆叠的自注意力层与逐位置全连接层，分别如图 1 左半部分和右半部分所示。

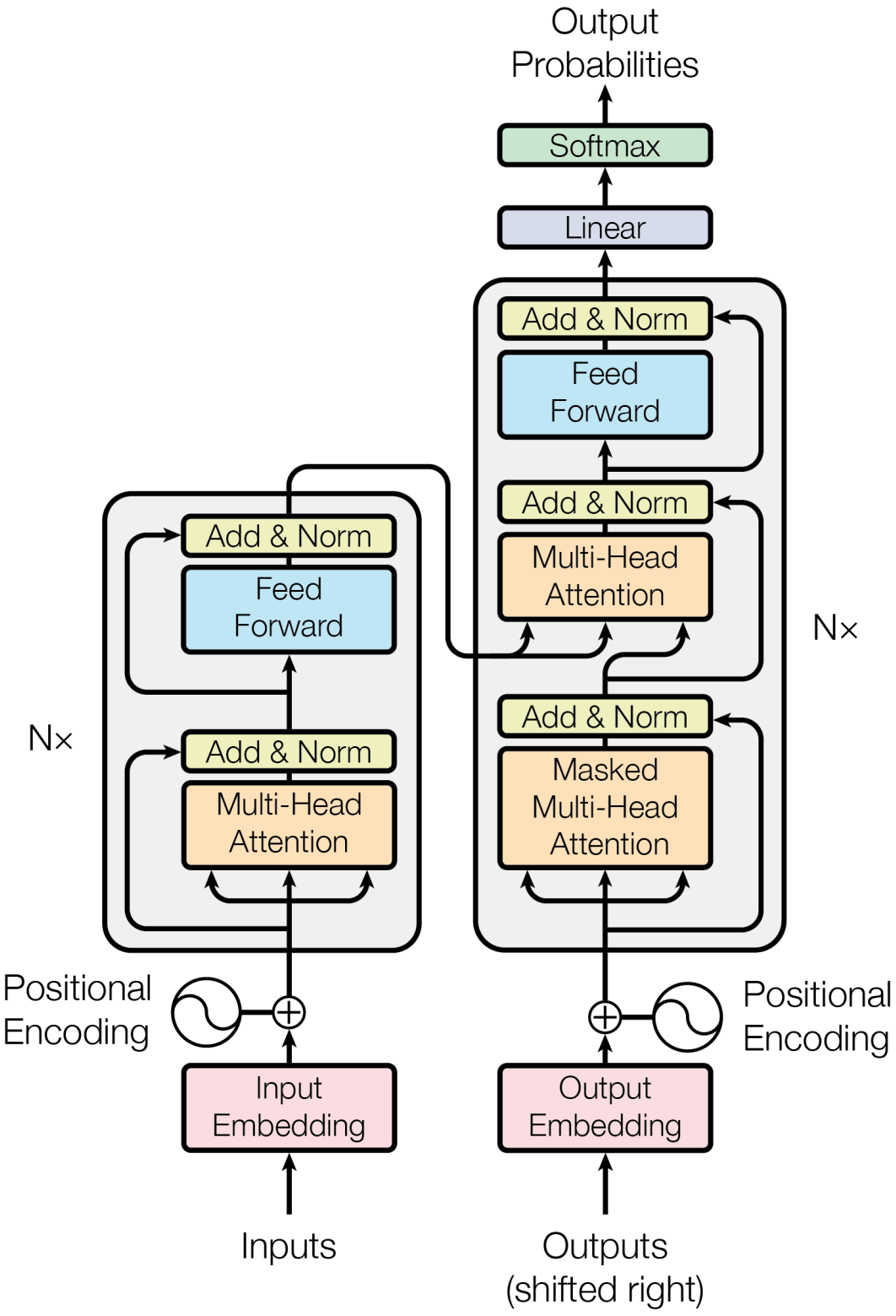


图 1 Transformer 模型架构。

3.1 编码器与解码器堆栈

编码器：编码器由 $N = 6$ 个相同层堆叠而成。每层包含两个子层：第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在两个子层周围均采用残差连接 [11]，随后进行层归一化 [1]。也就是说，每个子层的输出为 $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$，其中 $\mathrm{Sublayer}(x)$ 是子层自身实现的函数。为便于残差连接，模型中的所有子层以及嵌入层都产生维度为 $d_{\text{model}} = 512$ 的输出。

解码器：解码器同样由 $N = 6$ 个相同层堆叠而成。除每个编码器层中的两个子层外，解码器还插入第三个子层，对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个子层周围采用残差连接，随后进行层归一化。我们还修改了解码器堆栈中的自注意力子层，防止位置关注其后的位置。这种掩码与输出嵌入偏移一个位置相结合，确保对位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2 注意力

注意力函数可描述为：将一个查询和一组键—值对映射到一个输出，其中查询、键、值和输出均为向量。输出是值的加权和，而每个值所对应的权重由查询与相应键的兼容性函数计算得到。

3.2.1 缩放点积注意力

我们将这种特定注意力称为“缩放点积注意力”（图 2）。输入包括维度为 d_k 的查询和键，以及维度为 d_v 的值。我们计算查询与所有键的点积，将每个点积除以 $\sqrt{d_k}$，再应用 softmax 函数，得到各值的权重。

实践中，我们同时对一组查询计算注意力函数，并将这些查询打包成矩阵 Q。键和值也分别打包成矩阵 K 和 V。输出矩阵计算如下：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

两种最常用的注意力函数是加性注意力 [2] 和点积（乘性）注意力。除缩放因子 $1/\sqrt{d_k}$ 外，点积注意力与我们的算法相同。加性注意力使用具有单个隐藏层的前馈网络计算兼容性函数。尽管理论复杂度相近，点积注意力在实践中更快、更节省空间，因为它可用高度优化的矩阵乘法代码实现。

当 d_k 较小时，两种机制表现相近；当 d_k 较大时，加性注意力优于未缩放的点积注意力 [3]。我们推测，对于较大的 d_k，点积的量级会增大，使 softmax 函数进入梯度极小的区域。[1] 为抵消这一影响，我们将点积缩放 $1/\sqrt{d_k}$。

---

[1] 为说明点积为何会增大，假设 q 和 k 的各分量是均值为 0、方差为 1 的独立随机变量，则其点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0、方差为 d_k。

3.2.2 多头注意力

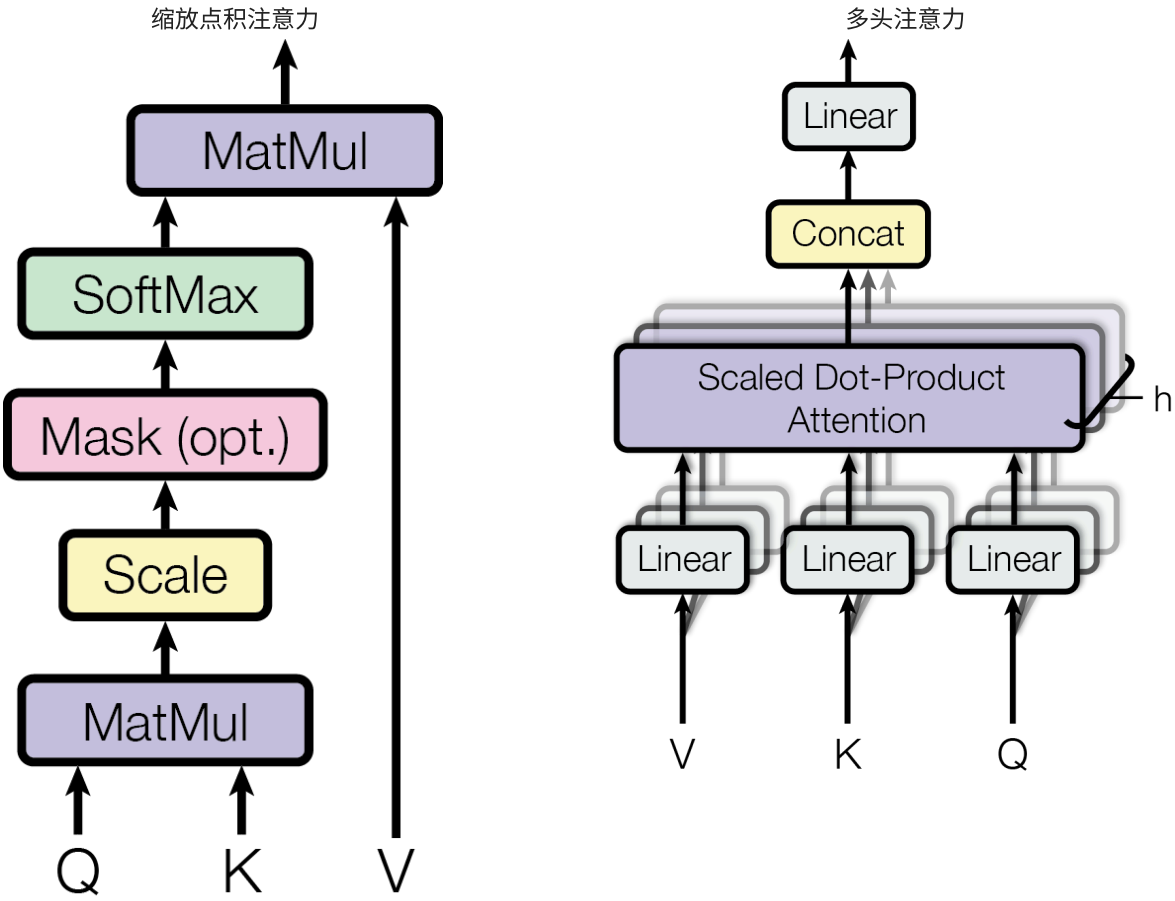


图 2 （左）缩放点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

我们发现，与其使用 d_{model} 维的键、值和查询执行一次注意力函数，不如通过不同的、学习得到的线性投影，将查询、键和值分别投影 h 次至 d_k、d_k 和 d_v 维。随后，在投影后的各组查询、键和值上并行执行注意力函数，得到 d_v 维输出值。将这些输出拼接并再次投影，即得到最终值，如图 2 所示。

多头注意力使模型能够联合关注不同位置、不同表示子空间中的信息。若只有一个注意力头，平均操作会抑制这种能力。

$$\begin{aligned}\text{MultiHead}(Q,K,V) &= \text{Concat}(\text{head}_1,\ldots,\text{head}_h)W^O\\ \text{其中}\quad \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)\end{aligned}$$

其中，投影所用参数矩阵为 $W_i^Q \in \mathbb{R}^{d_{\text{model}}\times d_k}$、$W_i^K \in \mathbb{R}^{d_{\text{model}}\times d_k}$、$W_i^V \in \mathbb{R}^{d_{\text{model}}\times d_v}$，以及 $W^O \in \mathbb{R}^{hd_v\times d_{\text{model}}}$。

本文采用 $h = 8$ 个并行注意力层，即 8 个注意力头。每个头使用 $d_k = d_v = d_{\text{model}}/h = 64$。由于每个头的维度降低，总计算成本与使用完整维度的单头注意力相近。

3.2.3 注意力在本模型中的应用

Transformer 以三种不同方式使用多头注意力：

- 在“编码器—解码器注意力”层中，查询来自前一个解码器层，记忆键和值来自编码器输出。这使解码器中的每个位置都能关注输入序列的所有位置，模拟了序列到序列模型中典型的编码器—解码器注意力机制，如文献 [38, 2, 9]。

- 编码器包含自注意力层。在自注意力层中，所有键、值和查询均来自同一处，此处即编码器中前一层的输出。编码器中的每个位置都可关注编码器前一层的所有位置。
- 类似地，解码器中的自注意力层允许解码器的每个位置关注解码器中直到该位置（含该位置）的所有位置。为保持自回归性质，需要阻止解码器中的左向信息流。我们在缩放点积注意力内部实现这一点：将 softmax 输入中所有对应非法连接的值屏蔽（设为 $-\infty$）。见图 2。

3.3 逐位置前馈网络

除注意力子层外，编码器和解码器中的每一层还包含一个全连接前馈网络，该网络分别且相同地应用于每个位置。它由两次线性变换组成，中间使用 ReLU 激活。

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

不同位置使用相同的线性变换，但不同层使用不同参数。另一种描述方式是将其视为两个卷积核大小为 1 的卷积。输入和输出维度为 $d_{\text{model}} = 512$，内层维度为 $d_{ff} = 2048$。

3.4 嵌入与 Softmax

与其他序列转导模型类似，我们使用学习得到的嵌入，将输入词元和输出词元转换为 d_{model} 维向量。我们还使用通常的线性变换和 softmax 函数，将解码器输出转换为预测的下一词元概率。与文献 [30] 类似，本模型在两个嵌入层与 pre-softmax 线性变换之间共享同一个权重矩阵。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$。

3.5 位置编码

由于模型不含循环和卷积，为使模型利用序列顺序，必须注入序列中词元的相对或绝对位置信息。为此，我们在编码器和解码器堆栈底部，将“位置编码”加入输入嵌入。位置编码与嵌入具有相同的 d_{model} 维，因此二者可以相加。位置编码有多种选择，可以是学习得到的，也可以是固定的 [9]。

本文使用不同频率的正弦和余弦函数：

$$\begin{aligned} PE_{(pos,2i)} &= \sin\left(pos / 10000^{2i/d_{\text{model}}}\right) \\ PE_{(pos,2i+1)} &= \cos\left(pos / 10000^{2i/d_{\text{model}}}\right) \end{aligned}$$

其中 pos 是位置，i 是维度。也就是说，位置编码的每个维度对应一条正弦曲线，波长构成从 2π 到 $10000 \cdot 2\pi$ 的几何级数。我们选择该函数，是因为假设它能让模型容易学习按相对位置进行关注：对于任何固定偏移 k，PE_{pos+k} 都可表示为 PE_{pos} 的线性函数。

我们还试验了改用学习得到的位置嵌入 [9]，发现两个版本的结果几乎相同（见表 3 的 (E) 行）。我们选择正弦版本，是因为它可能使模型能够外推到比训练中遇到的序列更长的序列。

4 为什么选择自注意力

本节比较自注意力层与常用于序列映射的循环层、卷积层。这里考虑的映射是将变长符号表示序列 $(x_1, \ldots, x_n)$ 转换为等长序列 $(z_1, \ldots, z_n)$，其中 $x_i, z_i \in \mathbb{R}^d$，例如典型序列转导编码器或解码器中的隐藏层。为说明采用自注意力的动机，我们考虑三项要求。

第一，每层的总计算复杂度。第二，可并行化的计算量，以所需最小顺序操作数衡量。

第三，网络中长程依赖之间的路径长度。学习长程依赖是许多序列转导任务的关键挑战。影响这种学习能力的一个关键因素，是前向与后向信号在网络中必须经过的路径长度。输入和输出序列任意位置组合之间的路径越

短，学习长程依赖就越容易 [12]。因此，我们还比较了由不同层类型构成的网络中，任意两个输入和输出位置之间的最大路径长度。

表 1 不同层类型的最大路径长度、每层复杂度和最小顺序操作数。n 为序列长度，d 为表示维度，k 为卷积核大小，r 为受限自注意力中的邻域大小。

层类型	每层复杂度	顺序操作数	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环层	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积层	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
自注意力（受限）	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

如表 1 所示，自注意力层以常数个顺序执行的操作连接所有位置，而循环层需要 $O(n)$ 个顺序操作。从计算复杂度看，当序列长度 n 小于表示维度 d 时，自注意力层快于循环层；在最先进机器翻译模型使用的句子表示中，情况通常如此，例如词片（word-piece）[38] 和字节对编码 [31] 表示。对于涉及很长序列的任务，为提高计算性能，可以将自注意力限制为仅考虑输入序列中以相应输出位置为中心、大小为 r 的邻域。这会将最大路径长度增至 $O(n/r)$。我们计划在未来工作中进一步研究这种方法。

卷积核宽度 \(k

自注意力还有一个附带好处：它可能产生更可解释的模型。我们检查了模型中的注意力分布，并在附录中展示和讨论示例。不仅各个注意力头明显学会了执行不同任务，许多注意力头似乎还表现出与句子句法和语义结构相关的行为。

5 训练

本节介绍模型的训练方案。

5.1 训练数据与批处理

我们使用标准的 WMT 2014 英德数据集训练，其中包含约 450 万个句对。句子采用字节对编码 [3]，使用约 37,000 个词元的源语言—目标语言共享词表。英法任务使用规模大得多的 WMT 2014 英法数据集，其中包含 3,600 万个句子，并将词元划分为 32,000 个词片的词表 [38]。句对按近似序列长度分批。每个训练批次包含一组句对，约含 25,000 个源词元和 25,000 个目标词元。

5.2 硬件与训练计划

我们在一台配有 8 块 NVIDIA P100 GPU 的机器上训练模型。对于采用本文所述超参数的基础模型，每个训练步骤约需 0.4 秒。基础模型共训练 100,000 步，即 12 小时。大模型（见表 3 最后一行）每步需 1.0 秒，共训练 300,000 步（3.5 天）。

5.3 优化器

我们使用 Adam 优化器 [20]，其中 $\beta_1 = 0.9$、$\beta_2 = 0.98$、$\epsilon = 10^{-9}$。训练过程中，学习率按以下公式变化：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5},\ step_num \cdot warmup_steps^{-1.5}) \tag{3}$$

这相当于在前 $warmup_steps$ 个训练步骤中线性提高学习率，此后按步数平方根的倒数成比例降低学习率。我们使用 $warmup_steps = 4000$。

5.4 正则化

训练期间采用三种正则化方法：

残差 Dropout：我们将 dropout [33] 应用于每个子层的输出，在其与子层输入相加并归一化之前执行。此外，在编码器和解码器堆栈中，我们还对嵌入与位置编码之和应用 dropout。基础模型的比率为 $P_{drop} = 0.1$。

标签平滑：训练期间，我们采用值为 $\epsilon_{ls} = 0.1$ 的标签平滑 [36]。这会损害困惑度，因为模型学会了更不确定，但能提高准确率和 BLEU 分数。

6 实验结果

6.1 机器翻译

表 2 在英译德和英译法 newstest2014 测试集上，Transformer 以仅为既有模型一小部分的训练成本，取得了优于此前最佳模型的 BLEU 分数。

模型	BLEU		训练成本（FLOPs）	
	英—德	英—法	英—德	英—法
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk 集成 [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL 集成 [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S 集成 [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer（基础模型）	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer（大模型）	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

在 WMT 2014 英译德任务上，Transformer 大模型（表 2 中的 Transformer (big)）比此前报告的最佳模型（包括集成模型）高出 2.0 BLEU 以上，以 28.4 BLEU 创造新的最佳结果。该模型配置见表 3 最后一行。训练使用 8 块 P100 GPU，耗时 3.5 天。即使是基础模型，也超过了此前发表的所有模型和集成模型，而训练成本仅为任何竞争模型的一小部分。

在 WMT 2014 英译法任务上，大模型取得 41.0 BLEU，超过此前发表的所有单模型，而训练成本不到此前最佳模型的四分之一。用于英译法的 Transformer 大模型采用 $P_{drop} = 0.1$ 的 dropout 比率，而非 0.3。

对于基础模型，我们将最后 5 个检查点平均得到单一模型，这些检查点每 10 分钟保存一次。对于大模型，则平均最后 20 个检查点。我们使用束搜索，束大小为 4，长度惩罚 $\alpha = 0.6$ [38]。这些超参数是在开发集上实验后选定的。推理时，最大输出长度设为输入长度 + 50，但可能时提前终止 [38]。

表 2 总结了实验结果，并将翻译质量和训练成本与文献中的其他模型架构进行比较。我们用训练时间、GPU 数量以及每块 GPU 持续单精度浮点算力的估计值三者相乘，估算训练模型所用的浮点运算数。[2]

[2] 对 K80、K40、M40 和 P100，我们分别采用 2.8、3.7、6.0 和 9.5 TFLOPS。

6.2 模型变体

表 3 Transformer 架构变体。未列出的值与基础模型相同。所有指标均在英译德开发集 newstest2013 上测得。所列困惑度为按字节对编码计算的每词片困惑度，不应与每词困惑度比较。

变体	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	训练步数	PPL（开发）	BLEU（开发）	参数 $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)		用位置嵌入替代正弦位置编码								4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

为评估 Transformer 各组成部分的重要性，我们以不同方式改变基础模型，并测量英译德开发集 newstest2013 上的性能变化。我们采用上一节所述的束搜索，但不进行检查点平均。结果见表 3。

在表 3 的 (A) 行中，我们改变注意力头数量以及注意力键和值的维度，同时按第 3.2.2 节所述保持计算量不变。单头注意力比最佳设置低 0.9 BLEU，但注意力头过多时质量也会下降。

在表 3 的 (B) 行中，我们观察到减小注意力键维度 d_k 会损害模型质量。这表明确定兼容性并不容易，采用比点积更复杂的兼容性函数可能有益。我们还在 (C) 和 (D) 行中观察到，不出所料，更大的模型效果更好，dropout 对避免过拟合非常有帮助。在 (E) 行中，我们用学习得到的位置嵌入 [9] 替代正弦位置编码，结果与基础模型几乎相同。

6.3 英语成分句法分析

表 4 Transformer 能很好地泛化到英语成分句法分析（结果为 WSJ 第 23 节）。

分析器	训练方式	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	仅 WSJ，判别式	88.3
Petrov et al. (2006) [29]	仅 WSJ，判别式	90.4
Zhu et al. (2013) [40]	仅 WSJ，判别式	90.4
Dyer et al. (2016) [8]	仅 WSJ，判别式	91.7
Transformer（4 层）	仅 WSJ，判别式	91.3
Zhu et al. (2013) [40]	半监督	91.3
Huang & Harper (2009) [14]	半监督	91.3
McClosky et al. (2006) [26]	半监督	92.1
Vinyals & Kaiser et al. (2014) [37]	半监督	92.1
Transformer（4 层）	半监督	**92.7**
Luong et al. (2015) [23]	多任务	93.0
Dyer et al. (2016) [8]	生成式	93.3

为评估 Transformer 是否能泛化到其他任务，我们进行了英语成分句法分析实验。该任务具有特定挑战：输出受到很强的结构约束，而且明显长于输入。此外，RNN 序列到序列模型在小数据条件下一直无法达到最佳结果 [37]。

我们在 Penn Treebank [25] 的《华尔街日报》（WSJ）部分训练了一个 $d_{model} = 1024$ 的 4 层 Transformer，训练句约 4 万句。我们还在半监督条件下训练，使用规模更大的高置信度语料和 BerkleyParser 语料，共约 1,700 万句 [37]。仅 WSJ 的设置使用 16K 词元词表，半监督设置使用 32K 词元词表。

我们仅在第 22 节开发集上进行了少量实验，以选择 dropout（包括注意力与残差 dropout，见第 5.4 节）、学习率和束大小；其他所有参数均保持英译德基础翻译模型不变。推理时，最大输出长度增至输入长度 + 300。仅 WSJ 和半监督两种设置均使用束大小 21 和 $\alpha = 0.3$。

表 4 的结果表明，尽管缺少针对任务的调优，模型仍表现出乎意料地好；除循环神经网络文法 [8] 外，它优于此前报告的所有模型。

与 RNN 序列到序列模型 [37] 不同，即使只在包含 4 万句的 WSJ 训练集上训练，Transformer 也优于 BerkeleyParser [29]。

7 结论

本文提出了 Transformer，这是首个完全基于注意力的序列转导模型，用多头自注意力取代编码器—解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度显著快于基于循环层或卷积层的架构。在 WMT 2014 英译德和英译法任务上，我们都取得了新的最佳结果。在前一任务中，我们的最佳模型甚至超过此前报告的所有集成模型。

我们对基于注意力的模型的未来感到振奋，并计划将其应用于其他任务。我们计划把 Transformer 扩展到文本以外的输入和输出模态，并研究局部、受限注意力机制，以高效处理图像、音频和视频等大型输入与输出。减少生成过程的顺序性也是我们的另一项研究目标。

用于训练和评估模型的代码见 https://github.com/tensorflow/tensor2tensor。

致谢：感谢 Nal Kalchbrenner 和 Stephan Gouws 富有成效的评论、修正与启发。

参考文献

1. Jimmy~Lei Ba, Jamie~Ryan Kiros, and Geoffrey~E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.
2. Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. CoRR, abs/1409.0473, 2014.
3. Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc~V. Le. Massive exploration of neural machine translation architectures. CoRR, abs/1703.03906, 2017.
4. Jianpeng Cheng, Li~Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. arXiv preprint arXiv:1601.06733, 2016.
5. Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. CoRR, abs/1406.1078, 2014.
6. Francois Chollet. Xception: Deep learning with depthwise separable convolutions. arXiv preprint arXiv:1610.02357, 2016.
7. Junyoung Chung, Caglar G\"ulcehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. CoRR, abs/1412.3555, 2014.
8. Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah~A. Smith. Recurrent neural network grammars. In Proc. of NAACL, 2016.
9. Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann~N. Dauphin. Convolutional sequence to sequence learning. arXiv preprint arXiv:1705.03122v2, 2017.
10. Alex Graves. Generating sequences with recurrent neural networks. arXiv preprint arXiv:1308.0850, 2013.
11. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 770--778, 2016.
12. Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and J\"urgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
13. Sepp Hochreiter and J\"urgen Schmidhuber. Long short-term memory. Neural computation, 9(8):1735--1780, 1997.
14. Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing, pages 832--841. ACL, August 2009.
15. Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. arXiv preprint arXiv:1602.02410, 2016.
16. ukasz Kaiser and Samy Bengio. Can active memory replace attention? In Advances in Neural Information Processing Systems, (NIPS), 2016.
17. ukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In International Conference on Learning Representations (ICLR), 2016.
18. Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van~den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. arXiv preprint arXiv:1610.10099v2, 2017.
19. Yoon Kim, Carl Denton, Luong Hoang, and Alexander~M. Rush. Structured attention networks. In International Conference on Learning Representations, 2017.
20. Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In ICLR, 2015.
21. Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. arXiv preprint arXiv:1703.10722, 2017.
22. Zhouhan Lin, Minwei Feng, Cicero Nogueira~dos Santos, Mo~Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. arXiv preprint arXiv:1703.03130, 2017.
23. Minh-Thang Luong, Quoc~V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. arXiv preprint arXiv:1511.06114, 2015.
24. Minh-Thang Luong, Hieu Pham, and Christopher~D Manning. Effective approaches to attention-based neural machine translation. arXiv preprint arXiv:1508.04025, 2015.
25. Mitchell~P Marcus, Mary~Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. Computational linguistics, 19(2):313--330, 1993.
26. David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In Proceedings of the Human Language Technology Conference of the NAACL, Main Conference, pages 152--159. ACL, June 2006.

27. Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In Empirical Methods in Natural Language Processing, 2016.

28. Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. arXiv preprint arXiv:1705.04304, 2017.

29. Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL, pages 433--440. ACL, July 2006.

30. Ofir Press and Lior Wolf. Using the output embedding to improve language models. arXiv preprint arXiv:1608.05859, 2016.

31. Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909, 2015.

32. Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.

33. Nitish Srivastava, Geoffrey~E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1):1929--1958, 2014.

34. Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C.~Cortes, N.~D. Lawrence, D.~D. Lee, M.~Sugiyama, and R.~Garnett, editors, Advances in Neural Information Processing Systems 28, pages 2440--2448. Curran Associates, Inc., 2015.

35. Ilya Sutskever, Oriol Vinyals, and Quoc~VV Le. Sequence to sequence learning with neural networks. In Advances in Neural Information Processing Systems, pages 3104--3112, 2014.

36. Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. CoRR, abs/1512.00567, 2015.

37. Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In Advances in Neural Information Processing Systems, 2015.

38. Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc~V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et~al. Google's neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144, 2016.

39. Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. CoRR, abs/1606.04199, 2016.

40. Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers), pages 434--443. ACL, August 2013.

附录：注意力可视化

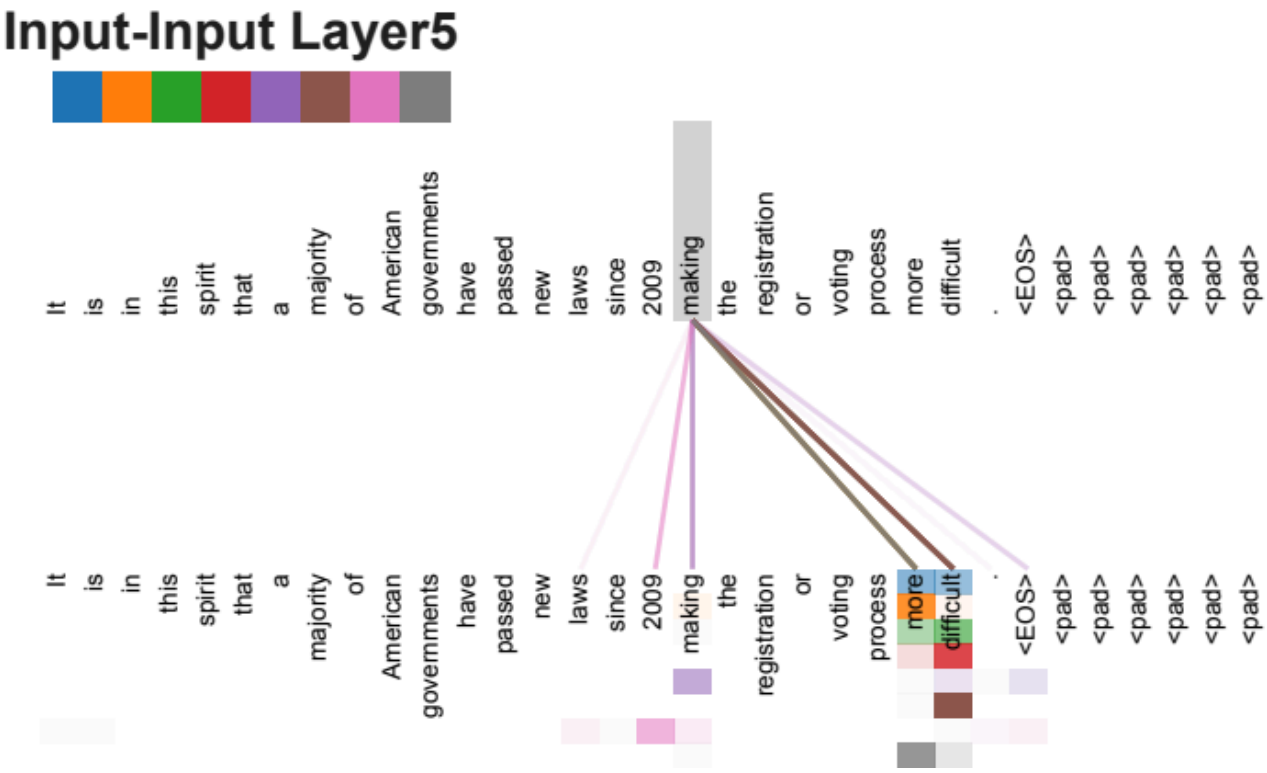


图 3 6 层编码器中第 5 层自注意力捕获长距离依赖的示例。许多注意力头关注动词“making”的远距离依赖，从而补全短语“making ... more difficult”。此处仅显示“making”一词的注意力。不同颜色表示不同注意力头。最好以彩色查看。

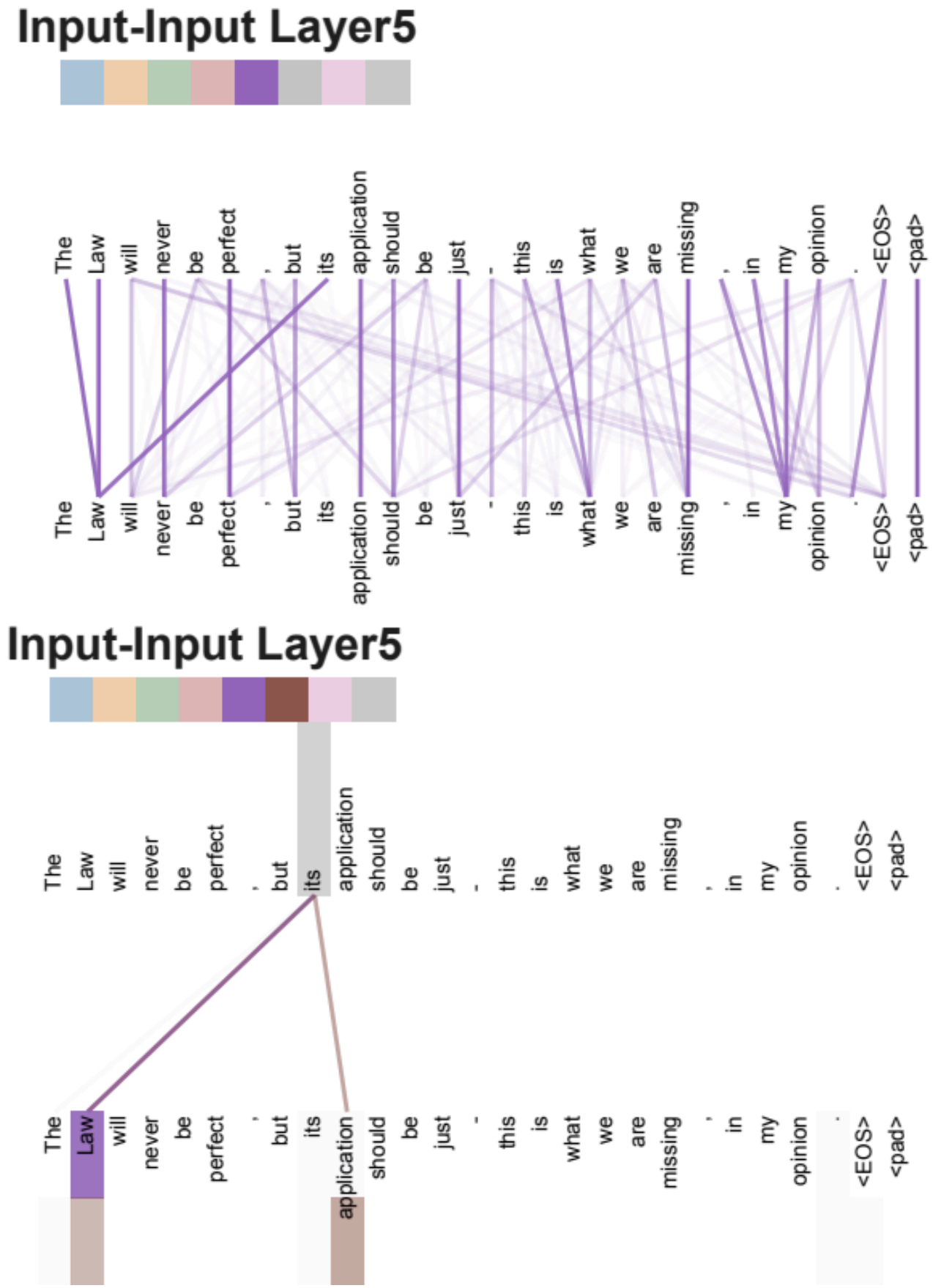


图 4 同样来自 6 层中第 5 层的两个注意力头，显然参与了指代消解。上：头 5 的完整注意力。下：仅显示单词“its”在注意力头 5 和 6 上的注意力。注意，该词的注意力分布非常尖锐。

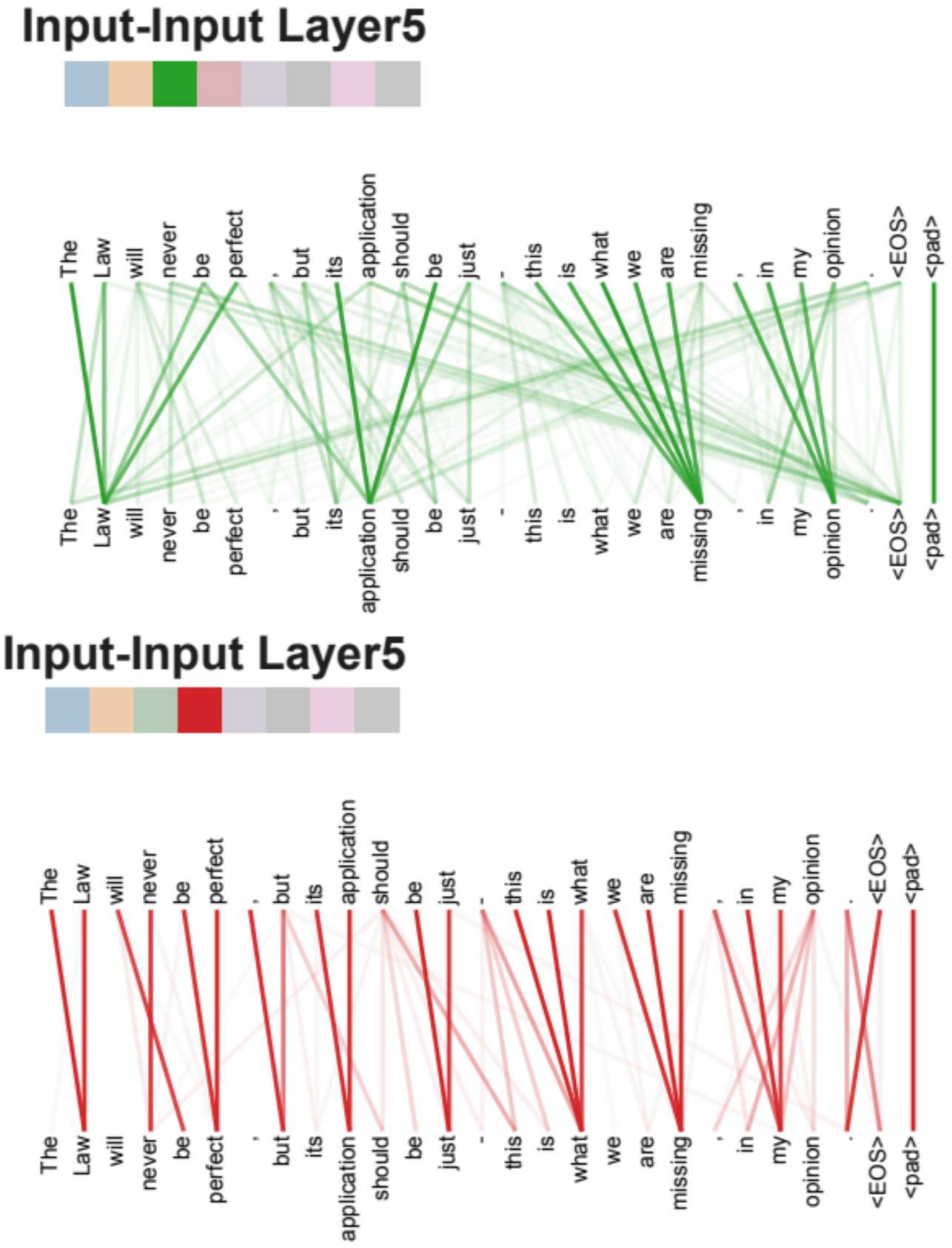


图 5 许多注意力头表现出的行为似乎与句子结构有关。上面给出两个示例，分别来自 6 层编码器中第 5 层自注意力的两个不同注意力头。这些头明显学会了执行不同任务。